

PostgreSQL Failover Procedures

Table of Contents

- [1. Learning Objectives](#)
- [2. Failover vs Switchover](#)
- [3. Architecture: Timeline Concepts](#)
- [4. Pre-Failover Checklist](#)
- [5. Manual Failover: `pg\_ctl promote`](#)
- [6. Timeline History and `pg\_rewind`](#)
- [7. Avoiding Split-Brain](#)
- [8. Automatic Failover with `pg\_auto\_failover`](#)
- [9. Post-Failover Steps](#)
- [10. Switchover \(Planned Failover\)](#)
- [11. Production Runbook](#)
- [12. Common Mistakes](#)
- [13. Best Practices](#)
- [14. Interview Questions](#)
- [15. Exercises and Solutions](#)

Learning Objectives

By the end of this module, you will be able to:

- Execute a manual failover with `pg_ctl promote`
- Understand PostgreSQL timeline history and its importance
- Use `pg_rewind` to re-sync the old primary as a new standby
- Identify and prevent split-brain scenarios
- Execute a planned switchover with zero data loss
- Write a production-grade failover runbook

Failover vs Switchover

Operation	Trigger	Data Loss Risk	Planned
Failover	Unplanned primary failure	Possible (async replication lag)	No
Switchover	Planned maintenance	None (fenced)	Yes

FAILOVER SCENARIO:

Time 0: Primary crashes unexpectedly
Time 1: Operations team detects outage (monitoring alert)
Time 2: Verify primary is truly dead (not network partition)
Time 3: Promote best standby to primary
Time 4: Redirect application connections
Time 5: Repair old primary, re-join as standby
Total downtime: 1-5 minutes (manual), 30-60 seconds (automated)

SWITCHOVER SCENARIO:

Time 0: Schedule maintenance window

```
Time 1: Stop writes to primary gracefully
Time 2: Wait for standby to fully catch up
Time 3: Promote standby, demote old primary
Time 4: Redirect application connections
Time 5: Old primary becomes new standby
Downtime: 10-30 seconds
```

Architecture: Timeline Concepts

When a standby is promoted, it creates a new **timeline**. Timelines prevent the old primary from accidentally rejoining and overwriting newer data.

```
INITIAL STATE:
Primary (Timeline 1)—WAL—> Standby1 (Timeline 1)
                    —WAL—> Standby2 (Timeline 1)

PRIMARY CRASHES at LSN 0/50000:
Timeline 1 ends at LSN 0/50000

STANDBY1 PROMOTED:
New Primary (Timeline 2) starts at 0/50001
└─ Has history: "Timeline 1 ended at 0/50000"

STANDBY2 (still on Timeline 1):
└─ Has WAL up to 0/48000 (was behind)
└─ Cannot rejoin Timeline 2 without pg_rewrite
    (its WAL diverges from Timeline 2 at 0/48000)

TIMELINE HISTORY FILE:
pg_wal/00000002.history:
"1\t0/50000\tno recovery target specified\n"
meaning: timeline 1 switched to timeline 2 at LSN 0/50000
```

```
TIMELINE DIAGRAM:

                                └─> Timeline 3 (if promoted again)
                                |
Timeline 1: —————|0/50000
Timeline 2 (new primary):      0/50000 —————>
Old standby2 (diverged at 0/48000): —————|
                                         needs pg_rewrite to follow Timeline 2
```

Pre-Failover Checklist

```
# Run these checks BEFORE promoting any standby

# 1. Confirm primary is truly unreachable (not just network partition)
ping -c 3 192.168.1.100
psql "host=192.168.1.100 port=5432 user=postgres connect_timeout=5" -c "SELECT 1;"
ssh postgres@192.168.1.100 "pg_ctl status -D /var/lib/postgresql/16/main"
```

```
# 2. Check which standby has the most recent data (highest LSN)
# Run on each standby:
psql -h 192.168.1.101 -U postgres -c "SELECT pg_last_wal_receive_lsn(),
pg_last_wal_replay_lsn();"
psql -h 192.168.1.102 -U postgres -c "SELECT pg_last_wal_receive_lsn(),
pg_last_wal_replay_lsn();"

# 3. Identify the most advanced standby (use this one!)
# Higher LSN = less data loss potential

# 4. Check replication lag at time of failure (from monitoring system)
# Look at last known replay_lag from pg_stat_replication monitoring

# 5. STOP the old primary if accessible (prevent split-brain)
ssh postgres@primary "sudo systemctl stop postgresql@16-main"
# or STONITH (Shoot The Other Node In The Head) via fencing device
```

Manual Failover: pg_ctl promote

Method 1: pg_ctl promote (PostgreSQL 12+)

```
# On the chosen standby server (postgres user)
# Replace /var/lib/postgresql/16/main with your actual data directory

# Option A: pg_ctl promote
sudo -u postgres pg_ctl promote -D /var/lib/postgresql/16/main

# Option B: touch the promote trigger file (older method)
sudo -u postgres touch /tmp/postgresql.trigger

# Option C: SQL function (if can connect to standby)
# Connect to the standby (read-only connection)
sudo -u postgres psql -c "SELECT pg_promote();"
# Returns true if promotion was triggered

# Verify promotion completed
sudo -u postgres psql -c "SELECT pg_is_in_recovery();"
# Should return: f (false = it's now primary)

# Check timeline incremented
sudo -u postgres psql -c "SELECT timeline_id FROM pg_control_checkpoint();"
# Should be 2 (if was timeline 1 before)
```

Verifying the New Primary

```
-- Connect to promoted server
-- Confirm it is no longer in recovery
SELECT pg_is_in_recovery();
```

```
-- f

-- Check current LSN
SELECT pg_current_wal_lsn();

-- Check timeline
SELECT timeline_id FROM pg_control_checkpoint();

-- Check it accepts writes
CREATE TABLE failover_test (id INT);
DROP TABLE failover_test;

-- If these succeed, server is writable = primary
```

Redirect Application Traffic

```
# Option 1: Update DNS / load balancer VIP
# Point application DNS record to new primary IP

# Option 2: HAProxy reconfiguration
# See 08_haproxy_load_balancing.md

# Option 3: Update application config
# Change DATABASE_URL to new primary IP
# Restart application servers

# Option 4: Use a connection proxy (PgBouncer)
# Update pgbouncer.ini to point to new primary
# Reload: psql -p 6432 -U pgbouncer pgbouncer -c "RELOAD;"
```

Timeline History and pg_rewind

After failover, the old primary and other standbys that were on the old timeline need to re-join. Use `pg_rewind` to efficiently re-sync without a full base backup.

What pg_rewind Does

```
Without pg_rewind:
Old primary has Timeline 1 WAL up to 0/50000 + uncommitted data
New primary has Timeline 2 from 0/50000
Old primary CANNOT join Timeline 2 without full re-sync

With pg_rewind:
pg_rewind rewinds old primary back to the common point (0/50000)
Discards diverged WAL
Old primary is now clean and can follow new primary (Timeline 2)
Only copies changed blocks – much faster than full backup
```

pg_rewind Steps

```

# Prerequisites: wal_log_hints = on OR data checksums on primary
# Check: SHOW wal_log_hints;

# Step 1: Stop the old primary (if still running)
sudo systemctl stop postgresql@16-main

# Step 2: Run pg_rewind
# Source = new primary, target = old primary data directory
sudo -u postgres pg_rewind \
    --target-pgdata=/var/lib/postgresql/16/main \
    --source-server="host=192.168.1.101 port=5432 user=postgres dbname=postgres" \
    --progress

# Output:
# servers diverged at WAL location 0/500000 on timeline 1
# rewinding from last common checkpoint at 0/4F0000 on timeline 1
# Done!

# Step 3: Configure old primary as new standby
cat > /var/lib/postgresql/16/main/postgresql.auto.conf << EOF
primary_conninfo = 'host=192.168.1.101 port=5432 user=replicator
application_name=old_primary'
EOF

# Step 4: Create standby.signal
touch /var/lib/postgresql/16/main/standby.signal

# Step 5: Start as standby
sudo systemctl start postgresql@16-main

# Step 6: Verify it's now a standby following new primary
sudo -u postgres psql -c "SELECT pg_is_in_recovery(), pg_last_wal_replay_lsn();"

```

pg_rewind Without Network Access

```

# If you can't connect to new primary, use WAL archive
sudo -u postgres pg_rewind \
    --target-pgdata=/var/lib/postgresql/16/main \
    --source-pgdata=/path/to/new_primary_data_directory \
    --progress

```

Avoiding Split-Brain

Split-brain occurs when two PostgreSQL instances both believe they are the primary and accept writes. This causes data divergence that is extremely difficult to reconcile.

How Split-Brain Happens

Scenario:

1. Primary P1 loses network connection to standbys
2. Standby S1 cannot reach P1, assumes P1 is dead
3. S1 is promoted to primary (Timeline 2)
4. P1 recovers network – it's STILL running as primary (Timeline 1)
5. Two primaries exist simultaneously!
6. Applications write to both → divergent data → catastrophe

Prevention Strategies

1. STONITH (Shoot The Other Node In The Head)

```
# Before promoting standby, fence the old primary
# Power off via IPMI/IDRAC
ipmitool -H primary-bmc-ip -U admin -P pass chassis power off

# Or via cloud API
aws ec2 stop-instances --instance-ids i-1234567890abcdef0

# Only promote after confirmed the old primary is powered off
```

2. Fencing via Reserved VIP

```
# Use keepalived/Pacemaker to control a virtual IP
# The VIP can only be held by one node
# If primary loses quorum, VIP is removed, app connections fail
# Standby gets VIP only after winning election
```

3. External Lock (DCS-based, Patroni approach)

Patroni prevents split-brain using distributed consensus:

1. Primary must continuously hold a "leader lock" in etcd/Consul
2. If primary cannot renew lock (network partition), it demotes itself
3. Another node can only acquire lock via DCS quorum
4. Only the lock holder is allowed to accept writes

4. Check Before Promoting

```
# Never promote without verifying old primary is confirmed dead
# Implement a "pause before promote" delay
# Require manual confirmation for failover in critical environments

function safe_promote() {
    STANDBY_HOST=$1

    # Check if primary is really down
    if pg_isready -h primary-host -p 5432 -t 5; then
        echo "ERROR: Primary is still responding! Aborting promotion."
        exit 1
    fi
}
```

```

fi

# Confirm via multiple paths
if ssh primary-host "pg_ctl status -D /var/lib/postgresql/16/main 2>/dev/null |
grep 'is running'"; then
    echo "ERROR: Primary PostgreSQL is still running! Aborting."
    exit 1
fi

echo "Primary confirmed down. Promoting $STANDBY_HOST..."
ssh $STANDBY_HOST "pg_ctl promote -D /var/lib/postgresql/16/main"
}

```

Post-Failover Steps

```

# 1. Verify new primary is healthy
psql -h new-primary -U postgres -c "SELECT version(), pg_is_in_recovery();"

# 2. Check for any replication slots on OLD primary that need cleanup
# (Old primary's slots are now orphaned)
# On new primary, check for dangling slots:
psql -h new-primary -c "SELECT slot_name, active FROM pg_replication_slots;"

# 3. Re-sync remaining standbys to new primary
# Each standby that was on the old timeline needs pg_rewind
for STANDBY in 192.168.1.102 192.168.1.103; do
    echo "Resyncing $STANDBY..."
    ssh postgres@$STANDBY "
        pg_ctl stop -D /var/lib/postgresql/16/main
        pg_rewind -D /var/lib/postgresql/16/main \
            --source-server='host=new-primary port=5432 user=postgres
dbname=postgres'
        echo 'primary_conninfo = ...' >> postgresql.auto.conf
        touch standby.signal
        pg_ctl start -D /var/lib/postgresql/16/main
    "
done

# 4. Verify all standbys reconnected
psql -h new-primary -c "SELECT application_name, state, sync_state FROM
pg_stat_replication;"

# 5. Update monitoring (Grafana, PagerDuty) to point to new primary

# 6. Create incident report
# 7. Schedule post-mortem

# 8. Verify application functionality
# Run smoke tests / health checks

```

Switchover (Planned Failover)

A switchover is a planned, graceful promotion with zero data loss.

```
# SWITCHOVER PROCEDURE

# Step 1: Check replication lag
psql -h primary -c "SELECT application_name, replay_lag FROM pg_stat_replication
WHERE application_name = 'target_standby';"
# Should be near zero

# Step 2: Stop application writes to primary
# e.g., maintenance mode, haproxy backend disable

# Step 3: Wait for standby to fully catch up
psql -h primary -c "
SELECT pg_current_wal_lsn() = flush_lsn AS standby_fully_synced
FROM pg_stat_replication
WHERE application_name = 'target_standby';"
# Wait until this returns: t

# Step 4: Confirm last WAL on primary matches standby
psql -h primary -c "SELECT pg_current_wal_lsn();"
psql -h standby -c "SELECT pg_last_wal_replay_lsn();"
# Both should match

# Step 5: Promote standby
ssh postgres@standby "pg_ctl promote -D /var/lib/postgresql/16/main"

# Step 6: Convert old primary to standby
# Stop old primary
ssh postgres@old-primary "pg_ctl stop -D /var/lib/postgresql/16/main"

# Configure as standby (no pg_rewind needed if caught up fully)
ssh postgres@old-primary "
cat > /var/lib/postgresql/16/main/postgresql.auto.conf << EOF
primary_conninfo = 'host=new-primary port=5432 user=replicator
application_name=old_primary'
EOF
touch /var/lib/postgresql/16/main/standby.signal
pg_ctl start -D /var/lib/postgresql/16/main"

# Step 7: Redirect application traffic to new primary
# Step 8: Verify
```

Production Runbook

```
=====
POSTGRESQL FAILOVER RUNBOOK
```


Version: 2.0 | Last Updated: 2024-01

=====

SEVERITY: P1 – Primary Database Down

ESCALATION:

On-call DBA → DB Team Lead → CTO (if > 30 min outage)

STEP 1: TRIAGE (0-2 minutes)

- a) Check monitoring dashboard: Is primary in DOWN state?
- b) Check primary server directly:
 - \$ ping -c 3 <primary-ip>
 - \$ psql -h <primary-ip> -U postgres -c "SELECT 1;" -c timeout=5
- c) Check application logs: What error are users seeing?
- d) Confirm this is NOT a network partition (can other services reach primary?)

STEP 2: ASSESS STANDBYS (2-4 minutes)

- a) SSH to each standby and check:
 - \$ sudo -u postgres psql -c "SELECT pg_is_in_recovery(), pg_last_wal_replay_lsn();"
- b) Identify standby with HIGHEST replay_lsn (least data loss)
- c) Note any replication lag: the behind standby will miss some commits

STEP 3: FENCE OLD PRIMARY (4-5 minutes)

- a) Attempt graceful shutdown:
 - \$ ssh postgres@<primary> "pg_ctl stop -D \$PGDATA -m fast"
- b) If unreachable, use STONITH:
 - \$ ipmitool -H <primary-bmc> chassis power off
 - OR
 - \$ aws ec2 stop-instances --instance-ids <instance-id>
- c) CONFIRM primary is offline before proceeding

STEP 4: PROMOTE STANDBY (5-6 minutes)

- a) SSH to best standby
- b) Promote:
 - \$ sudo -u postgres pg_ctl promote -D \$PGDATA
 - OR
 - \$ sudo -u postgres psql -c "SELECT pg_promote();"
- c) Verify:
 - \$ sudo -u postgres psql -c "SELECT pg_is_in_recovery();"
 - Must return 'f'

STEP 5: REDIRECT TRAFFIC (6-8 minutes)

- a) Update HAProxy/PgBouncer to point to new primary
- b) Update DNS record (if using DNS for failover)
- c) Notify application team to update connection strings if needed

STEP 6: VERIFY (8-10 minutes)

```
a) Run smoke test from application server:
  $ psql -h <new-primary> -U appuser -d appdb -c "SELECT COUNT(*) FROM
critical_table;"
```

- ```
b) Check application health endpoints
c) Monitor error rates in APM (Datadog, New Relic)
```

```
STEP 7: STABILIZE (10+ minutes)
```

---

- ```
a) Pg_rewind old primary as new standby
b) Verify all standbys are replicating from new primary
c) Update monitoring thresholds
d) Send status update to stakeholders
```

```
ROLLBACK PLAN:
```

```
If new primary has issues, reverse the process:
```

- ```
- Stop new primary
- pg_rewind old primary (if recovered)
- Promote old primary
```

```
=====
```

---

## Common Mistakes

1. **Promoting without confirming old primary is dead** — causes split-brain
2. **Promoting the wrong standby** — not checking which has the most recent WAL
3. **Forgetting to run pg\_rewind** on old primary — it has diverged timeline
4. **Not updating application connection strings** — application still tries old primary
5. **Not testing failover** — first time is during an actual outage
6. **Skipping the post-failover checklist** — orphaned replication slots cause disk issues
7. **Not documenting the incident** — repeated mistakes
8. **Using pg\_ctl promote on the wrong server** — accidentally promoting a replica

---

## Best Practices

1. **Practice failover quarterly** — automate drills in staging
2. **Use Patroni** for automatic, safe failover in production
3. **Always fence first, promote second** — prevent split-brain
4. **Have a VIP or DNS-based failover mechanism** for application transparency
5. **Keep pg\_rewind enabled** ( `wal_log_hints = on` ) for fast re-sync
6. **Monitor promotion readiness** — know which standby is most current at all times
7. **Maintain a written runbook** — keep it updated and tested

---

## Interview Questions

**Q1: What is the difference between failover and switchover?**

A: Failover is an unplanned promotion due to primary failure; it may involve data loss equal to replication lag. Switchover is planned and graceful: writes are stopped, standby catches up fully, then promotion happens with zero data loss. Both involve promoting a standby to primary, but switchover is controlled and safe.

**Q2: What is split-brain and how do you prevent it?**

A: Split-brain is when two nodes both believe they are the primary, each accepting writes. It causes data divergence. Prevention: (1) STONITH — physically power off the old primary before promoting standby. (2) DCS-based leader locks (Patroni/etcd) — old primary self-demotes if it can't renew the lock. (3) Fencing via VIP — only one node can hold the virtual IP.

### Q3: Walk through the process of using `pg_rewind` to re-sync an old primary.

A: After failover, the old primary has WAL that diverged from the new primary's timeline. Steps: (1) Stop old primary. (2) Run `pg_rewind --target-pgdata=<old-primary> --source-server=<new-primary>`. It identifies the divergence point and copies only changed blocks from new primary. (3) Configure as standby with `primary_conninfo` pointing to new primary. (4) Create `standby.signal`. (5) Start — it will follow new primary. Requires `wal_log_hints = on` or data checksums.

### Q4: How do you determine which standby to promote after primary failure?

A: Check `pg_last_wal_replay_lsn()` on each standby. The one with the highest LSN has applied the most WAL and will have the least data loss. Automated tools like Patroni track this continuously and always promote the most advanced candidate.

### Q5: What happens to replication slots after failover?

A: Slots are stored on the primary's disk. After failover, the new primary doesn't have the old primary's slots. Standbys that were using slots on the old primary need new slots on the new primary. Old primary's slot (if it somehow comes back as standby) is orphaned and must be dropped.

### Q6: What is a timeline in PostgreSQL and why does it matter?

A: A timeline is a monotonically increasing integer that identifies a distinct history of WAL records. When a standby is promoted, it increments the timeline. This allows multiple diverged WAL streams to coexist in the archive without ambiguity. It prevents old primary from accidentally reusing WAL positions and ensures `pg_rewind` can identify exactly where histories diverge.

### Q7: Can you reuse `pg_basebackup` instead of `pg_rewind` after failover?

A: Yes, but `pg_rewind` is much faster. `pg_basebackup` copies the entire database (~hundreds of GB), while `pg_rewind` only copies changed blocks since the divergence point. For large databases, `pg_rewind` reduces re-sync time from hours to minutes.

### Q8: What is `pg_promote()` and how is it different from `pg_ctl promote` ?

A: Both trigger standby promotion. `pg_ctl promote` is a server control command run on the OS shell. `pg_promote()` is a SQL function (PostgreSQL 12+) that triggers promotion via a database connection — useful when you can connect to the standby but not SSH to it.

---

## Exercises and Solutions

### Exercise 1: Failover Simulation

```
#!/bin/bash
Simulate a failover on a test environment

PRIMARY=192.168.1.100
STANDBY=192.168.1.101
```

```

echo "Step 1: Checking current state..."
psql -h $PRIMARY -U postgres -c "SELECT pg_is_in_recovery(), pg_current_wal_lsn();"
psql -h $STANDBY -U postgres -c "SELECT pg_is_in_recovery(),
pg_last_wal_replay_lsn();"

echo "Step 2: Simulating primary failure..."
ssh postgres@$PRIMARY "pg_ctl stop -D /var/lib/postgresql/16/main -m immediate"

echo "Step 3: Verifying primary is down..."
if pg_isready -h $PRIMARY -p 5432 -t 5; then
 echo "ERROR: Primary is still up!"
 exit 1
fi

echo "Step 4: Promoting standby..."
ssh postgres@$STANDBY "pg_ctl promote -D /var/lib/postgresql/16/main"
sleep 3

echo "Step 5: Verifying promotion..."
psql -h $STANDBY -U postgres -c "SELECT pg_is_in_recovery();"

echo "Failover simulation complete!"

```

## Exercise 2: Post-Failover Health Check

```

-- Run on new primary after failover

-- 1. Confirm this is the primary
SELECT pg_is_in_recovery() AS is_standby;

-- 2. Check timeline changed
SELECT timeline_id FROM pg_control_checkpoint();

-- 3. Check standbys are connecting
SELECT application_name, state, sync_state, replay_lag
FROM pg_stat_replication;

-- 4. Check for orphaned slots
SELECT slot_name, active, restart_lsn
FROM pg_replication_slots
WHERE active = false;

-- 5. Verify writes work
BEGIN;
INSERT INTO failover_audit (event, ts) VALUES ('failover_test', now());
ROLLBACK;

```

## Cross-References

- [02 streaming replication.md](#) — Streaming replication setup

- [06\\_patroni.md](#) — Automated failover with Patroni
- [08\\_haproxy\\_load\\_balancing.md](#) — Traffic routing during failover
- [09\\_ha\\_architecture\\_patterns.md](#) — Complete HA architectures